

Coding Standards Document

Budget It by Dev OOPS

Contents

1. INTRODUCTION
2. REPOSITORY STRUCTURE
3. CODING CONVENTIONS
4. TOOLING AND CONFIGURATION
5. TESTING STANDARDS
6. DOCUMENTATION STANDARDS
7. PULL REQUEST GUIDELINES
8. CONTINUOUS INTEGRATION

1. INTRODUCTION

This document showcases the coding standards, conventions, and practices adopted for the development of the mobile budgeting app. These standards ensure that all developers follow the same style, resulting in code that is uniform, consistent, readable, and maintainable. The coding standards improve reliability and reduce bugs and inconsistencies. Code should be supported by meaningful comments where necessary and organized into clear widgets, functions, and modules to improve efficiency and maintainability.

2. REPOSITORY STRUCTURE

- Flutter/budgetit/lib/database/ – Database setup, schema, and DAO access.
- Flutter/budgetit/lib/database/daos/ – DAO files such as transaction_dao.dart, category_dao.dart, and budget_dao.dart.
- Flutter/budgetit/lib/models/ – Application data models.
- Flutter/budgetit/lib/services/ – Business logic and application services.
- Flutter/budgetit/lib/shared/widgets/ – Reusable widgets used across multiple screens.
- Flutter/budgetit/lib/views/ – Screen/page-level UI.
- Feature folders: views/dashboard/, views/budget_manager/, views/transaction_manager/, views/graphical_reports/.
- Flutter/budgetit/lib/utils/ – Shared helpers, themes, colours, icons, and formatting utilities.
- Flutter/budgetit/assets/ – Images, fonts, and static assets.
- Flutter/budgetit/test/widget/ – Widget tests.
- Flutter/budgetit/test/unit/ – Unit tests.
- Flutter/budgetit/test/support/ – Test helpers and fixtures.
- Flutter/budgetit/test/unit/database/ – DAO tests.
- Flutter/budgetit/test/unit/services/ – Service tests.
- Flutter/budgetit/test/unit/analysis/ – Analysis service tests.

3. CODING CONVENTIONS

- Language: Dart
- Framework: Flutter
- Database: Drift
- State Management: Provider
- Testing: Flutter Test, Mockito, in-memory Drift database
- Use flutter_lints.
- Run dart format before committing.
- Keep analyzer warnings clean.
- Files: snake_case.dart
- Classes/Widgets: PascalCase
- Variables/Functions: camelCase
- Private members: prefix with _
- Enums: PascalCase type with lowerCamelCase values.
- Screens in lib/views/<feature_name>/; reusable widgets in lib/shared/widgets/; DAOs in lib/database/daos/; business logic in lib/services/; models in lib/models/; utilities in lib/utils/.
- Use context.colours for colours and reuse shared widgets.
- Use DAOs for database operations instead of querying Drift directly from widgets.
- Test DAOs/services under test/unit and widgets under test/widget.
- Indentation: 2 spaces.
- Prefer final and const.
- Use async/await.
- Keep build methods readable.

4. TOOLING AND CONFIGURATION

- Dart formatter: dart format .
- Analyze: flutter analyze (or .fvm/flutter_sdk/bin/flutter.bat analyze).
- Dependencies configured in pubspec.yaml including flutter, drift, provider, decimal, mockito, flutter_test, and flutter_lints.
- Assets declared under flutter/assets.
- Generate Drift code using: dart run build_runner build
- For conflicts: dart run build_runner build --delete-conflicting-output

5. TESTING STANDARDS

- Use flutter_test for widget tests.
- Use unit tests for DAOs, services, and business logic.
- Use widget tests for reusable widgets and screens.
- Use integration tests for complete user flows.
- Use in-memory Drift databases for testing.

- Store fixtures in test/support/.
- Run tests using FVM and generate coverage.

6. DOCUMENTATION STANDARDS

- Write concise comments where necessary.
- Use Dart documentation comments for public APIs.
- Keep README.md and database documentation up to date.
- Avoid outdated comments.
- Do not manually edit generated files.

7. PULL REQUEST GUIDELINES

- Format code before submission.
- Run flutter analyze.
- Run relevant tests.
- Run build_runner after Drift schema changes.
- Use clear commit messages.
- Keep PRs focused.
- Do not merge failing code.
- Do not commit failing code.
- Mention where you used AI and request peer review.

8. CONTINUOUS INTEGRATION

- GitHub Actions is used for CI.
- CI runs on feature, development, and main branches.
- CI performs Flutter setup, dependency installation, code generation, analysis, and testing.
- Coverage is uploaded to Codecov.
- SonarCloud performs code quality checks.
- Main workflow includes golden, performance, and integration tests.